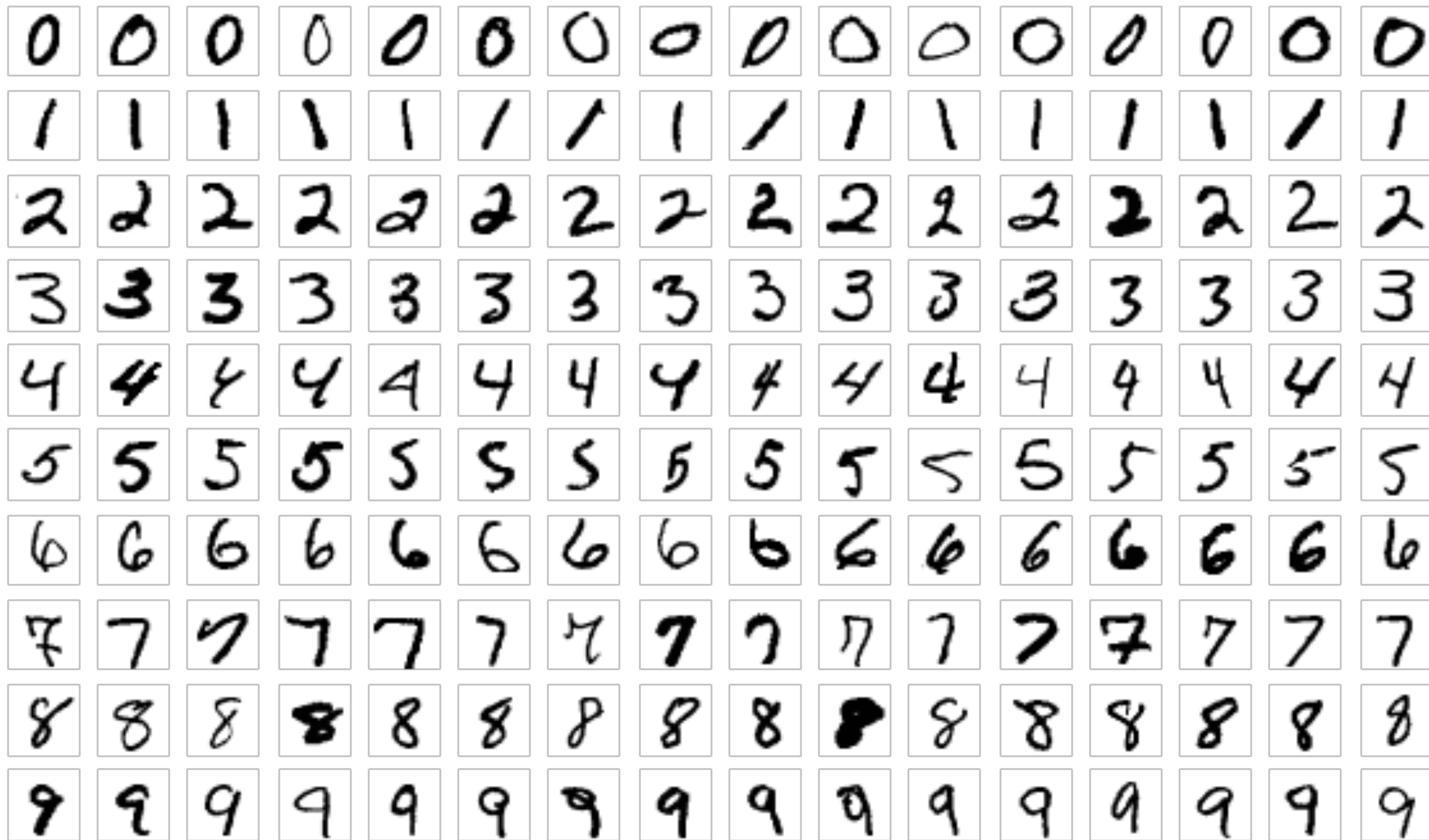


PyTorch로 MNIST 데이터셋 분류 모델 구현



MNIST 데이터셋



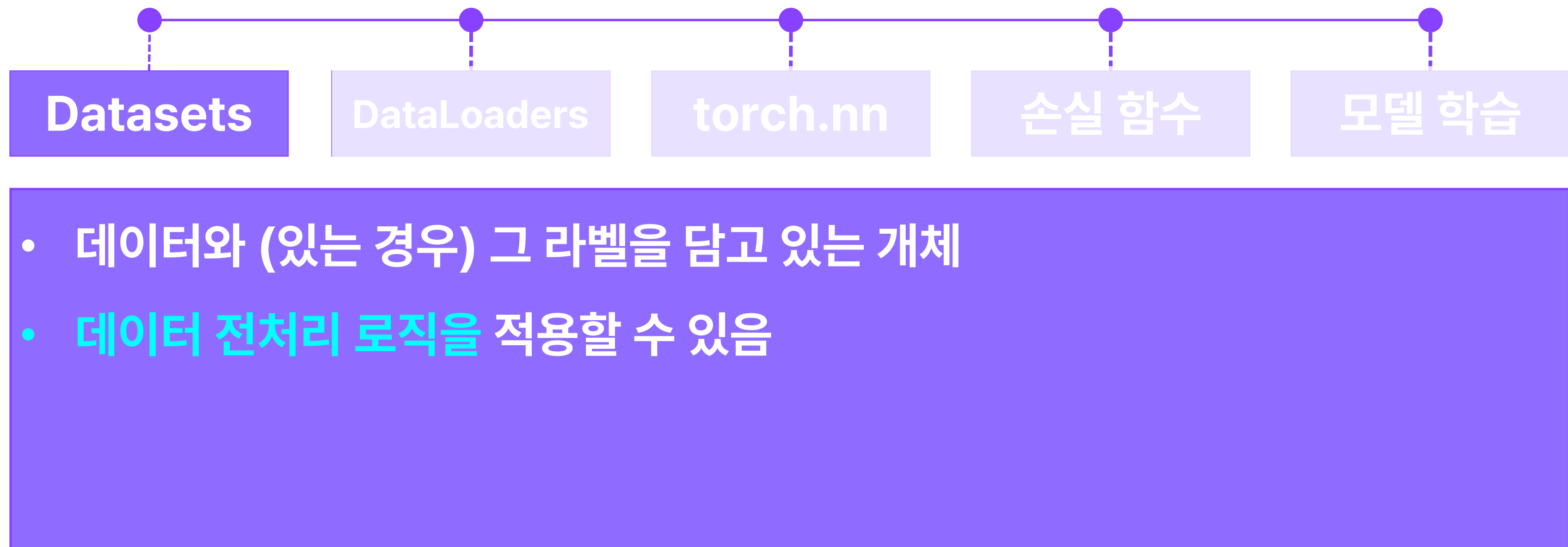
손글씨 숫자 이미지 데이터셋

손으로 쓴 0 ~ 9 까지의 숫자를

올바르게 분류하는 Task

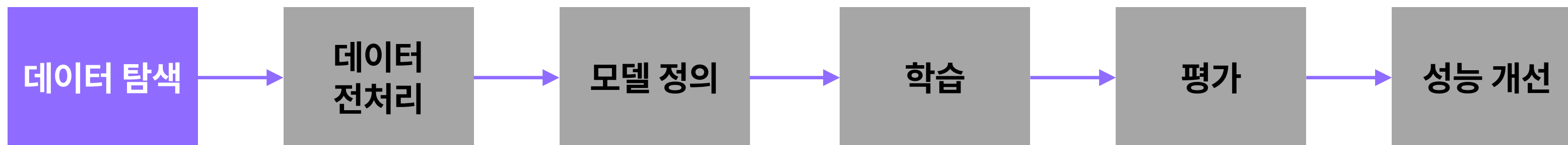


PyTorch 주요 키워드





MNIST 데이터셋 탐색



- MNIST 데이터셋은 PyTorch의 **torchvision** 라이브러리에서 불러오는 기능을 지원한다.

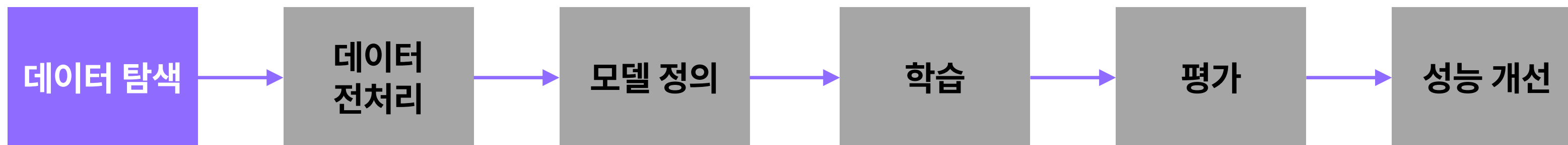
```
from torchvision.datasets import MNIST
from torchvision.transforms import ToTensor

transform = ToTensor()

# Load datasets
train_dataset = MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = MNIST(root='./data', train=False, download=True, transform=transform)
```




MNIST 데이터셋 탐색



- MNIST 데이터셋은 PyTorch의 **torchvision** 라이브러리에서 불러오는 기능을 지원한다.
 - **MNIST 데이터셋의 경우** PyTorch 자체적으로 학습, 테스트 데이터셋을 별도로 불러올 수 있다.
 - torchvision 라이브러리에서 기본 지원하는 다른 데이터셋은 공식 문서를 확인할 수 있다.
 - <https://pytorch.org/vision/main/datasets.html>

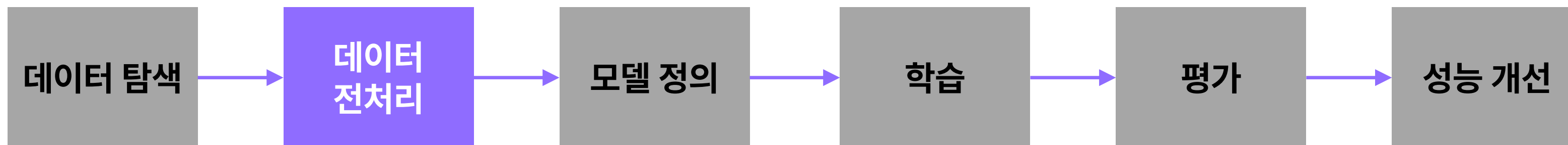
```
from torchvision.datasets import MNIST
from torchvision.transforms import ToTensor

transform = ToTensor()

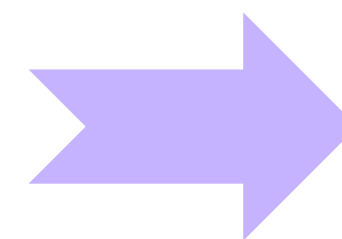
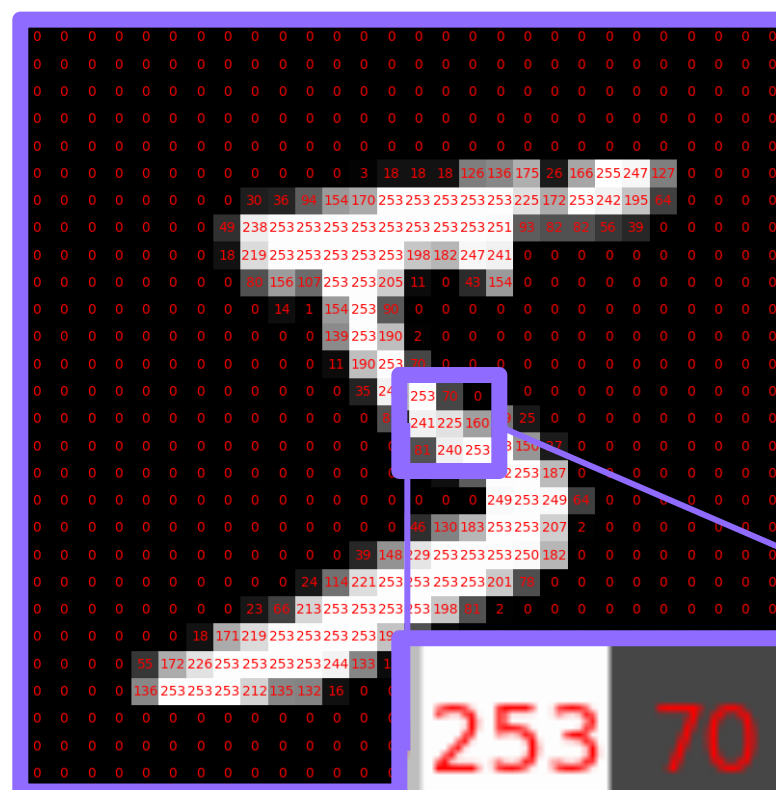
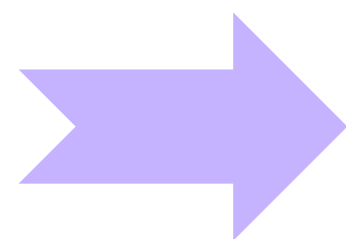
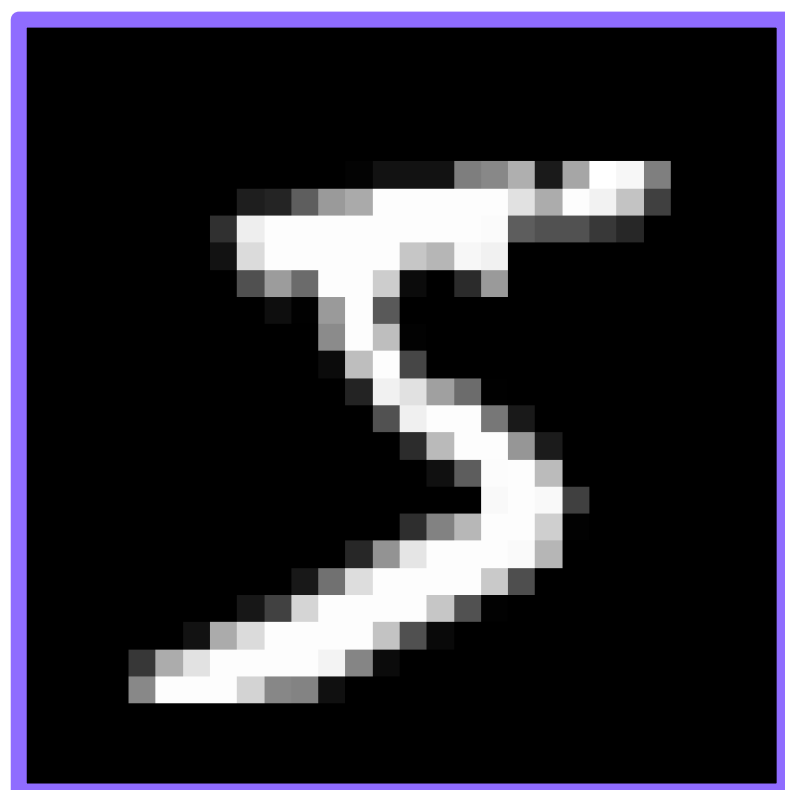
# Load datasets
train_dataset = MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = MNIST(root='./data', train=False, download=True, transform=transform)
```



MNIST 데이터 전처리



- 이미지 데이터를 일렬로 나열된 픽셀 값 배열로 변환한다.

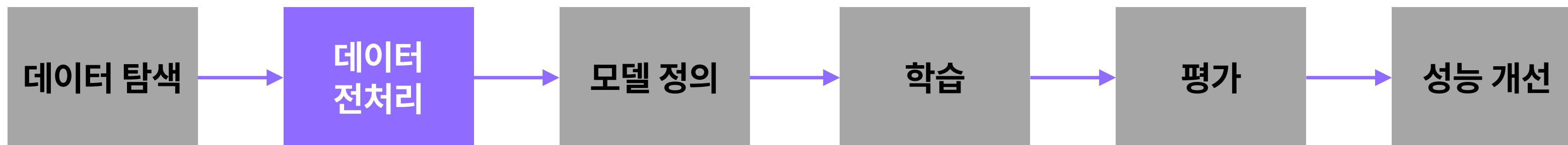


⋮
253
70
0
⋮





MNIST 데이터 전처리



- MNIST 데이터셋은 PyTorch의 **torchvision** 라이브러리에서 불러오는 기능을 지원한다.
- 이번 실습에서는 PyTorch 모델이 처리할 수 있도록 데이터의 픽셀 값을 Tensor로 변환하는 전처리만 적용하지만, 필요에 따라 다양한 전처리 기법을 적용할 수 있다.

```
from torchvision.datasets import MNIST
from torchvision.transforms import ToTensor

transform = ToTensor()

# Load datasets
train_dataset = MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = MNIST(root='./data', train=False, download=True, transform=transform)
```



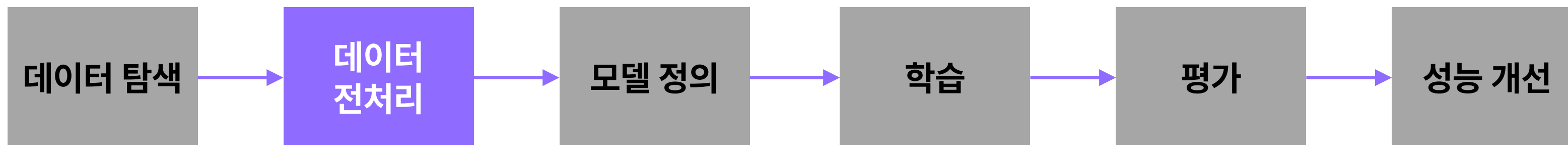
검증 (Validation) 데이터



학습 과정에서 검증을 위해 학습 데이터의 일부를 검증 데이터로 분리하여 사용
일반적으로 학습 데이터의 10~20%를 사용



MNIST Dataset 분할



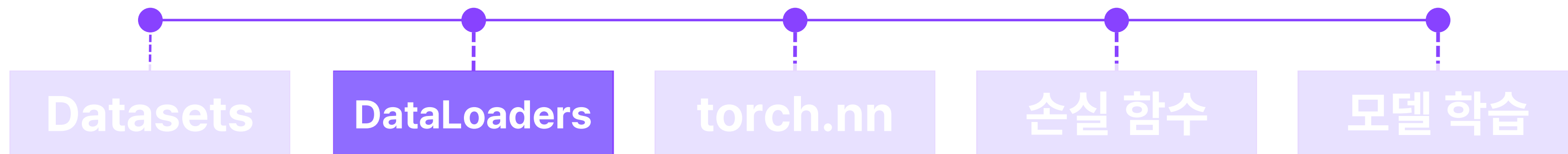
- PyTorch의 random_split 함수를 활용해서 Dataset을 분할할 수 있다.
- MNIST 학습 데이터셋 6만개 중 랜덤하게 1만개는 검증 Dataset으로 분할하여 저장한다.

```
from torch.utils.data import random_split

val_size = 10000
train_data, val_data = random_split(train_dataset, [len(train_dataset) - val_size, val_size])
```



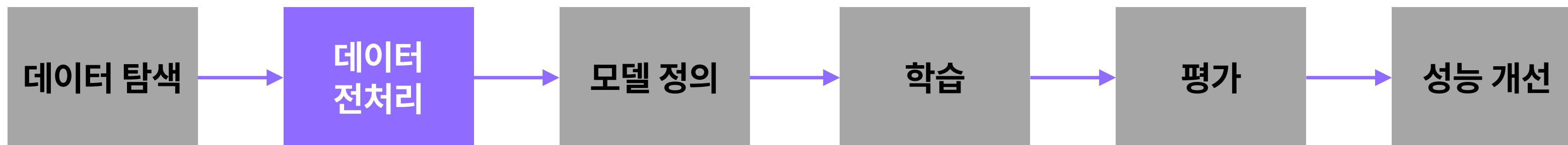
PyTorch 주요 키워드



- 학습 및 테스트 과정에서 **Dataset을 순회하면서** 데이터를 제공하는 개체
- Mini-batch 구성 등 데이터셋에 대한 다양한 로직을 적용할 수 있음.



MNIST DataLoader 생성



- Dataset을 활용해서 학습 과정에서 데이터를 불러올 수 있도록 **DataLoader를 생성한다.**

```
from torch.utils.data import DataLoader
```

```
# DataLoader 구성
```

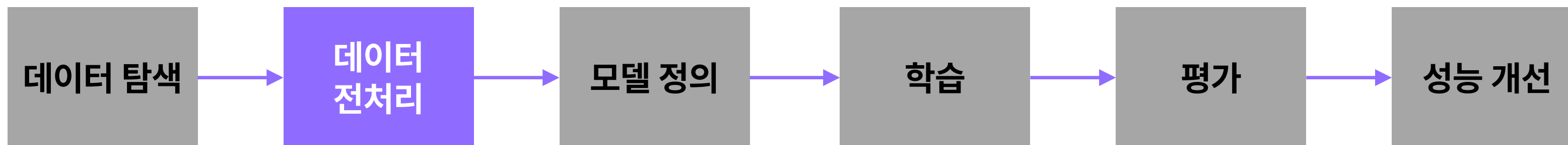
```
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
```

```
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)
```

```
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```



MNIST DataLoader 생성



- Dataset을 활용해서 학습 과정에서 데이터를 불러올 수 있도록 **DataLoader**를 생성한다.
- DataLoader는 인자로 입력 받은 **Dataset**에서 **mini-batch 단위로** 데이터를 순회한다.

```
train_data, val_data = random_split(train_dataset, [len(train_dataset) - val_size, val_size])
```

```
from torch.utils.data import DataLoader
```

```
# DataLoader 구성
```

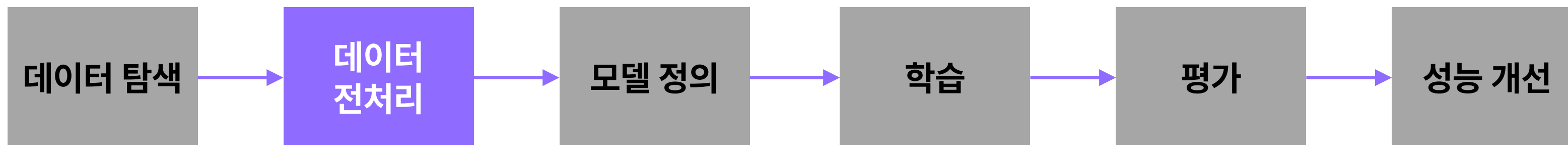
```
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
```

```
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)
```

```
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```




MNIST DataLoader 생성



- Dataset을 활용해서 학습 과정에서 데이터를 불러올 수 있도록 **DataLoader**를 생성한다.
- DataLoader는 인자로 입력 받은 **Dataset**에서 **mini-batch** 단위로 데이터를 순회한다.
- mini-batch의 크기는 **batch_size** 인자를 통해 조절할 수 있다.

```
train_data, val_data = random_split(train_dataset, [len(train_dataset) - val_size, val_size])
```

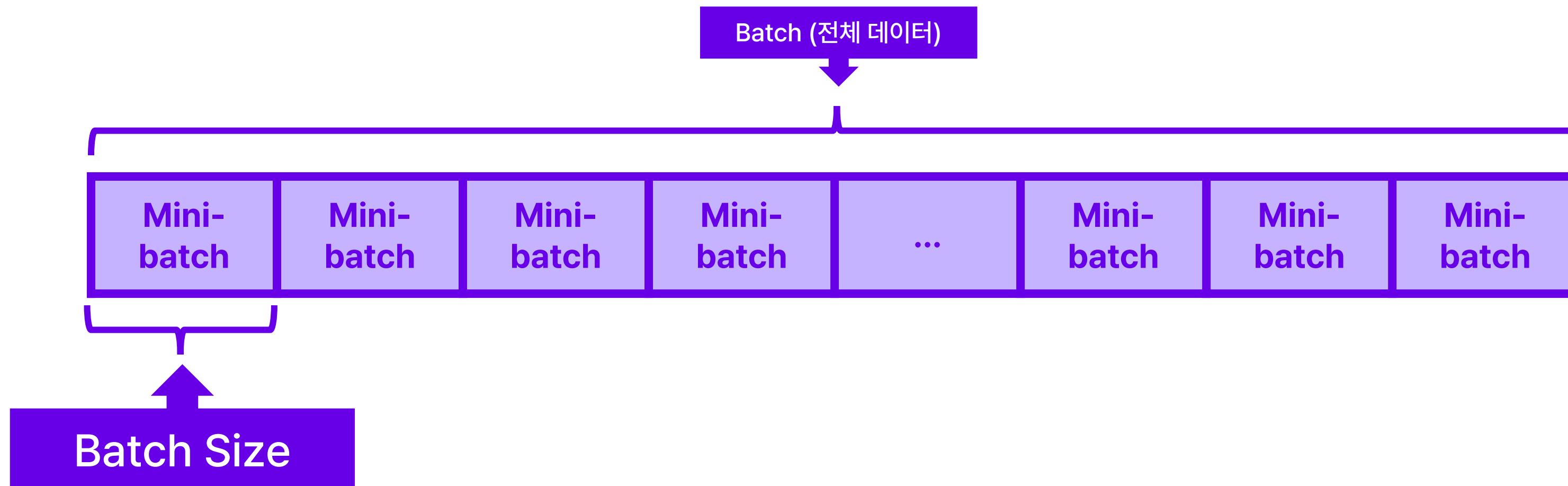
```
from torch.utils.data import DataLoader

# DataLoader 구성
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```



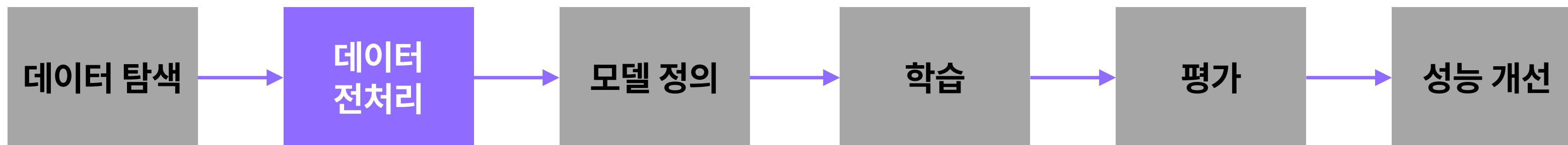
Batch size

- 매 기울기 조정에 **일부 학습 데이터 (mini-batch)**를 활용함으로써 학습을 최적화한다.
- 한 mini-batch의 크기를 Batch size 라고 칭한다.
- 대부분의 딥러닝 학습에서 mini-batch 방식을 사용하기 때문에, batch는 보통 엄밀한 의미의 mini-batch를 칭하는 경우가 많다. 이로 인해 mini-batch size 대신 batch size로 표현한다.





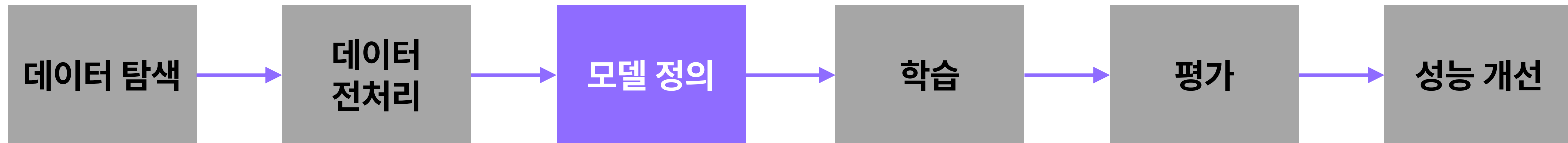
MNIST DataLoader 생성



- Dataset을 활용해서 학습 과정에서 데이터를 불러올 수 있도록 **DataLoader**를 생성한다.
- DataLoader는 인자로 입력 받은 **Dataset**에서 **mini-batch** 단위로 데이터를 순회한다.
- mini-batch의 크기는 **batch_size** 인자를 통해 조절할 수 있다.
- **shuffle** 인자를 통해 Dataset에 있는 데이터 순서대로 순회할지, 랜덤하게 순회할지를 결정한다. 학습을 위한 DataLoader를 제외하면 순서대로 순회하는 것이 좋다.

```
from torch.utils.data import DataLoader

# DataLoader 구성
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

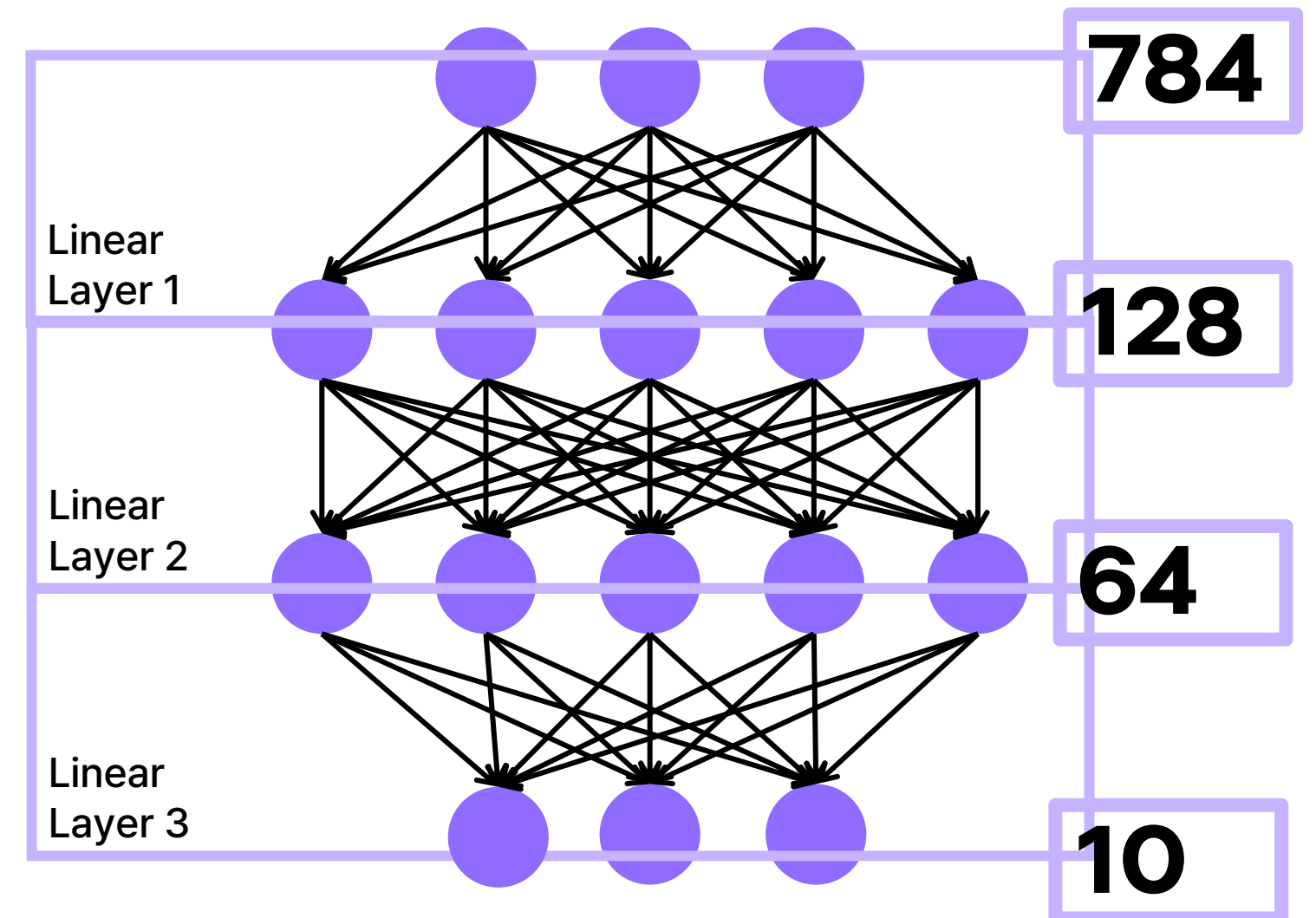


- MLP 모델 구조를 정의한다.

```
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)

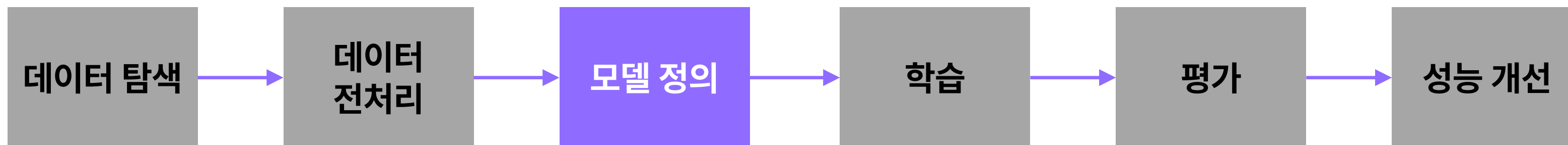
    def forward(self, x):
        x = self.flatten(x)
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.fc3(x)
        return x

model = MLP()
```





검증 데이터셋 분할



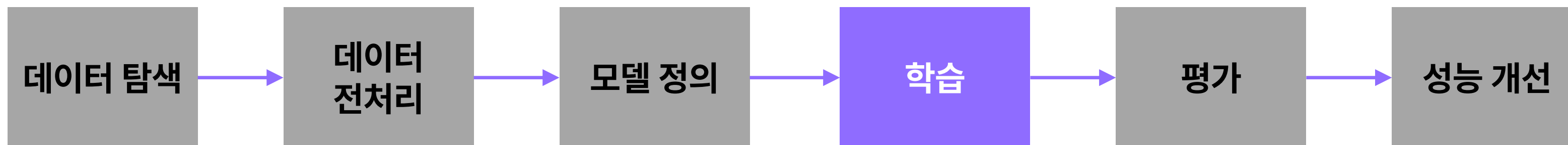
- 모델 학습 과정에서 사용할 최적화 함수(optimizer), 손실 함수(loss)를 설정한다.

```
import torch.nn as nn
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```



검증 데이터셋 분할



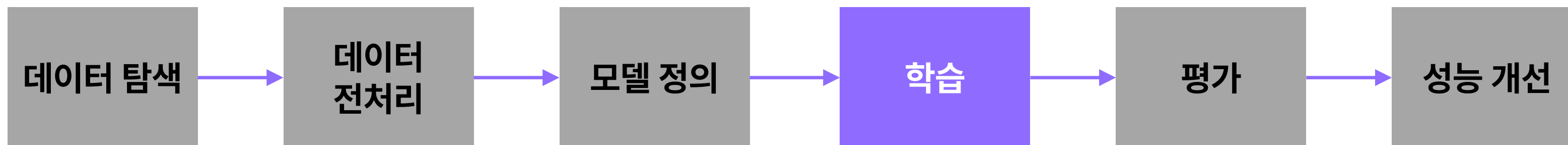
- 모델을 **MNIST 데이터에 학습시키고**, 매 epoch 마다 검증 결과를 확인한다.

학습 코드

```
epochs = 10
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader: # DataLoader
        optimizer.zero_grad() # 최적화 함수 초기화
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        loss = criterion(outputs, labels) # 모델의 예측값과 참값을 바탕으로 손실 (Loss) 계산
        loss.backward() # 손실 함수의 역전파 연산
        optimizer.step() # 최적화 함수를 통한 모델 업데이트
```



검증 데이터셋 분할



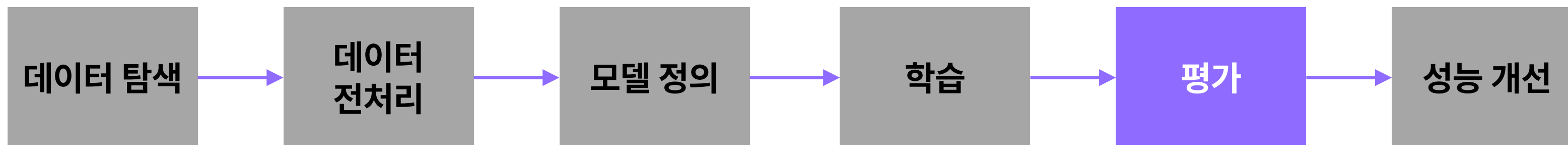
- 모델을 MNIST 데이터에 학습시키고, **매 epoch** 마다 검증 결과를 확인한다.
- 학습 및 검증 로직을 직접 작성함에 유의한다.

```
# 학습 코드와 같은 for loop에서 각 epoch 마다 실행되는 검증 코드
model.eval() # 모델을 검증 모드로 설정
val_loss = 0
correct = 0
with torch.no_grad(): # 최적화를 진행하지 않도록 환경 설정
    for images, labels in val_loader: # DataLoader
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        val_loss += criterion(outputs, labels).item()
        pred = outputs.argmax(dim=1, keepdim=True)
        correct += pred.eq(labels.view_as(pred)).sum().item()
```

검증 코드



검증 데이터셋 분할



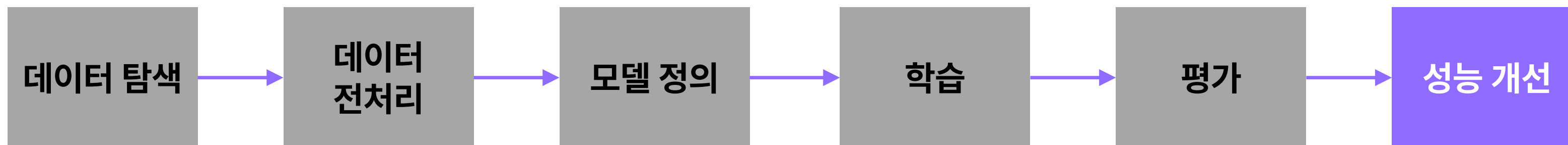
- 학습을 마쳤으면 최종 평가를 진행한다.

```
# 5. 모델 평가
model.eval()
test_loss = 0
correct = 0
with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        test_loss += criterion(outputs, labels).item()
        pred = outputs.argmax(dim=1, keepdim=True)
        correct += pred.eq(labels.view_as(pred)).sum().item()

test_loss /= len(test_loader)
test_acc = correct / len(test_loader.dataset)
```




검증 데이터셋 분할



- 학습이 완료된 모델에 대해서는 러닝 레이트, 배치 사이즈, 에포크 수를 조정해볼 수 있다.
- 정규화나 데이터 증강, 드롭아웃 같은 방법을 적용해 보는 등 추가적으로 성능을 개선해볼 수 있다.